

Inheriting Models, Extending Logic & API Integration

A beginner friendly walkthrough of model inheritance, computed and stored fields, and calling an external JSON API from Python, ending in a hands on ISBN validation feature.

`_inherit`

`@api.depends`

`store=True`

`requests.get()`

`ISBN check`

Where We Left Off, Where We're Going



Quick Recap

- Module 5: built a custom addon from scratch, models, views, and security.
- Module 6: customized the website builder and basic eCommerce pages.
- Today we stop building new models and start extending models that already exist.



Today's Agenda

- 1 Inheriting Existing Models
- 2 Extending Business Logic
- 3 Computed Fields
- 4 Stored Fields
- 5 API Integration (requests + JSON)
- 6 Hands On: ISBN Validation

Inheriting Existing Models

Community Edition gives us one clean tool for this: classic inheritance with `_inherit`.

Plain Language

`_inherit` lets us open up a model someone else already built, like `product.template`, and quietly add our own fields or methods to it. No copying, no new table.

`_name` vs `_inherit`

- `_name` creates a brand new model and table.
- `_inherit` reopens an existing model in place.
- Community only supports this classic style, no delegation model.

models/library_book.py

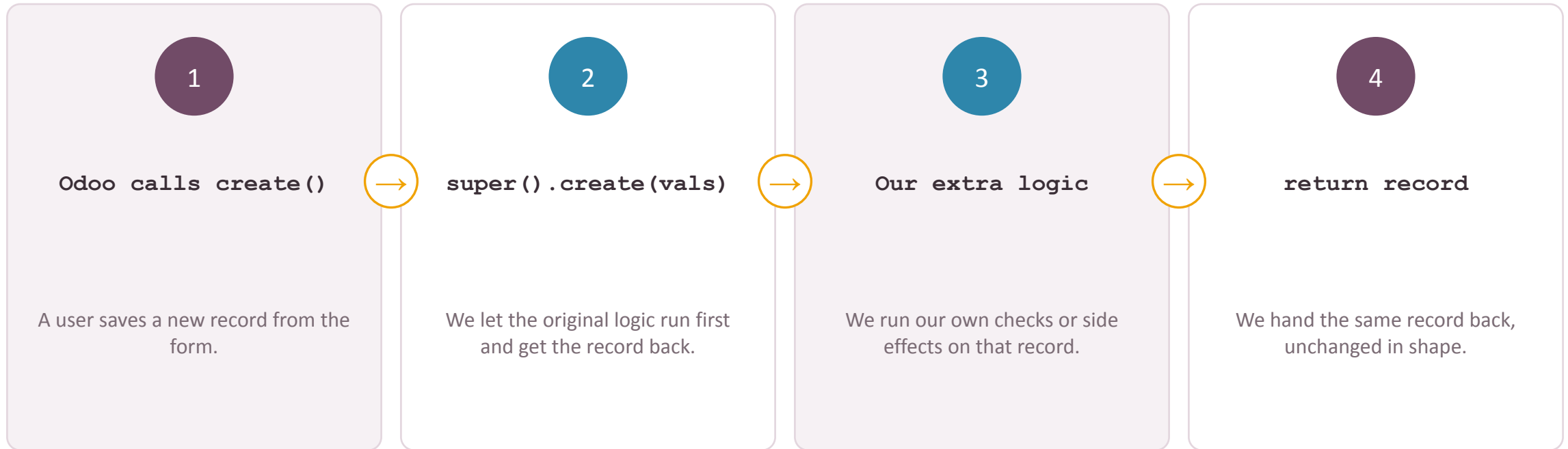
```
from odoo import models, fields

class LibraryBook(models.Model):
    _inherit = "product.template"

    isbn = fields.Char(
        string="ISBN",
        help="13 digit book code"
    )
```

Extending Business Logic

Adding fields is only half the story. Now we teach existing methods new behavior.



Warning: Always call `super()` unless you deliberately want to replace the original behavior. Forgetting it is the single most common beginner mistake in overrides.

Computed Fields

A computed field fills itself in by running a method, instead of a human typing a value.

Two pieces work together

- `@api.depends("field_a")` tells Odoo which fields to watch.
- `compute="_compute_x"` points the field to the method that fills it in.
- Whenever a watched field changes, Odoo reruns the method automatically.
- You never call this method yourself, Odoo calls it for you.

models/library_book.py

```
isbn_valid = fields.Boolean(  
    string="ISBN Valid",  
    compute="_compute_isbn_valid",  
)  
  
@api.depends("isbn")  
def _compute_isbn_valid(self):  
    for record in self:  
        record.isbn_valid = bool(  
            record.isbn and len(record.isbn) == 13  
        )
```

Stored Fields: store=True

A computed field can live only in memory, or be saved permanently in the database.

Not Stored (default)

- Recomputed every time it is read
- Cannot be used in search filters
- Cannot be grouped or sorted on
- Good for cheap, simple calculations

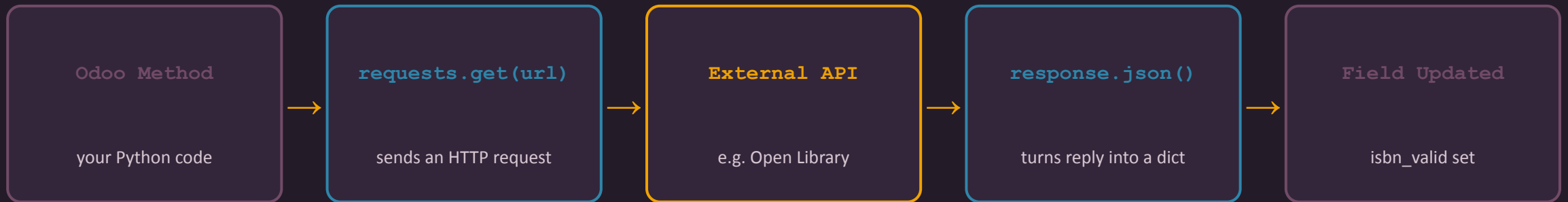
store=True

- Saved to the database like a normal field
- Fully searchable and filterable
- Usable in group by and reports
- Recomputed only when a dependency changes

Rule of thumb: if you will ever search, filter, or group by this field, add store=True.

API Integration: requests + JSON

Odoo can reach outside itself and talk to any web service that speaks JSON.



```
models/library_book.py

import requests

def _fetch_isbn_data(self, isbn):
    url = "https://openlibrary.org/api/books"
    params = {"bibkeys": f"ISBN:{isbn}", "format": "json"}
    response = requests.get(url, params=params, timeout=5)
    if response.status_code != 200:
        return {}
    return response.json()
```

Hands On: ISBN Validation

Everything today comes together in one feature: a self checking ISBN field.

1

Add the isbn field

A plain Char field on our inherited model, where students type the book code.

2

Add isbn_valid, stored

A computed boolean field with store=True so it can be filtered on later.

3

Call the external API

The compute method sends the ISBN to Open Library using requests.get().

4

Add a manual check button

A button that lets a user re trigger the lookup on demand.

Tip: By the end of class, typing a real ISBN into the form and saving should flip isbn_valid to true automatically.

Building It, Step by Step

1 Extend the model

Add `_inherit = "product.template"` in a new file inside `models/`.

2 Declare the fields

`isbn` as Char, `isbn_valid` as Boolean with `compute` and `store=True`.

3 Write the compute method

Decorate with `@api.depends("isbn")` and call the API inside it.

4 Handle failures gracefully

Wrap the request in `try/except` and default `isbn_valid` to `False`.

5 Add the field to the form view

Inherit the product form view and place the new fields beside the name.

6 Test with a real ISBN

Save a record and confirm the checkbox updates without a manual refresh.

Common Pitfalls

These are the mistakes almost every beginner makes on their first try.

Danger: Never call an external API inside a loop over thousands of records. It will time out and can get your IP blocked.

Warning: Forgetting `store=True` means the field looks fine on screen but cannot be searched, filtered, or grouped.

Warning: Not setting a timeout on `requests.get()` can freeze the whole save action if the external API is slow.

Tip: Wrap every external call in `try/except` and fall back to `isbn_valid = False` rather than raising an error.

What You Can Do Now

- Extend any existing Odoo model safely with `_inherit`.
- Override a method and still respect the original with `super()`.
- Build computed fields, and decide when they should be stored.
- Call an external JSON API from Python and handle its response.